

Architecture logicielle & systèmes distribués

[Accueil](#)[Introduction](#)[Jour 1](#)[Jour 2](#)[Jour 3](#)[Jour 4](#)[Jour 5](#)[Jour 6](#)[TP1](#)[TP2](#)[TP3](#)[Exercices](#)[Corrigés](#)[Glossaire](#)[Live Coding](#)[Dossier Architecture](#)[FAQ](#)[Support HTML modulaire](#)[Master 2](#)[6 jours](#)[3 TP progressifs](#)[Java / Spring Boot](#)

Introduction au module

Objectifs pédagogiques

Comprendre la philosophie du cours, le projet fil rouge, les ADR et la progression pédagogique.

Temps estimé

≈ 1h30 (lecture + quiz)

Prérequis

Bases Java / Spring Boot, intérêt pour la conception logicielle

Table des matières

- [Positionnement](#)
- [Pourquoi l'archi ?](#)
- [Objectifs de compétence](#)
- [Fil rouge](#)
- [ADR](#)
- [Dette technique](#)
- [Méthode pédagogique](#)
- [Parcours junior → expert](#)

- [Quiz d'entrée](#)

Positionnement du module

Ce module ne forme pas seulement à "coder des services". Il apprend à **penser un système**, à expliciter des compromis et à choisir une architecture adaptée au contexte. La maîtrise du code seul ne suffit plus dès qu'une équipe grandit et qu'un système doit vivre plusieurs années.

Définition

Une **architecture logicielle** décrit l'organisation globale d'un système : ses blocs principaux, leurs responsabilités, leurs échanges, les contraintes structurantes et les règles d'évolution. C'est l'ensemble des *décisions difficiles à défaire*.

Mode enfant 🧸

Quand tu ranges ta chambre, tu sépares les vêtements, les livres et les jouets dans des tiroirs différents. Si tout est mélangé dans un seul tas, chaque fois que tu cherches quelque chose tu dois tout remuer et tu risques de casser ce que tu cherchais. En logiciel, c'est exactement pareil.

Mode pro 🎓

L'architecture répond aux **attributs de qualité** (ISO 25010) : maintenabilité, évolutivité, résilience, sécurité, observabilité, exploitabilité et alignement avec l'organisation des équipes (Loi de Conway). Une architecture qui ne répond à aucun attribut de qualité explicite est probablement accidentelle.

"Architecture is the decisions that you wish you could get right early in a project." — Ralph Johnson

Pourquoi l'architecture, concrètement ?

La question revient souvent : *"Pourquoi ne pas juste coder et voir ce qui se passe ?"* Voici ce qui arrive quand on ne pense pas l'architecture.

Ce qui arrive sans architecture ❌

- **Big Ball of Mud** : tout dépend de tout, personne ne comprend plus le système après 6 mois.
- **Régression permanente** : toucher une feature en casse une autre non liée.
- **Tests impossibles** : la logique métier est mélangée avec la BD et l'HTTP.
- **Déploiements risqués** : chaque mise en production est une opération à haut risque.
- **Onboarding long** : il faut des semaines à un nouveau développeur pour être productif.

Exemple réel — Knight Capital Group (2012)

Knight Capital a perdu **440 millions de dollars en 45 minutes** à cause d'un déploiement bâclé sur un système mal architecturé. Un ancien code de test, jamais supprimé, s'est activé en production sur certains serveurs. Sans monitoring, sans rollback propre, sans isolation des modules, l'incident était inévitable.

L'architecture logicielle, c'est aussi de la **gestion du risque opérationnel**.

Ce qu'une bonne architecture permet ✅

- Modifier une feature sans toucher au reste.
- Tester chaque couche indépendamment.
- Déployer en confiance, avec rollback possible.
- Scaler uniquement la partie sous charge.
- Onboarder un nouveau développeur en 2 jours.

Objectifs de compétence

À la fin des 6 jours, vous serez capables de :

Compétence	Niveau visé	Évalué dans
Lire et critiquer une architecture existante	Autonome	Dossier architecture
Structurer un monolithe propre (couches, DDD léger)	Autonome	TP 1
Comprendre les compromis du distribué	Guidé → Autonome	TP 2
Concevoir des APIs robustes et versionnées	Autonome	TP 2 + 3
Documenter les décisions via ADR	Guidé	Dossier architecture
Mettre en place observabilité + résilience	Guidé	TP 3

Erreur fréquente ●

Vouloir commencer par Kubernetes, Kafka ou un service mesh sans avoir clarifié le domaine métier, les flux et les qualités cibles. Les outils servent une architecture — ils n'en sont pas une.

Fil rouge — Campus Library

Nous partons d'un **monolithe propre** de gestion de bibliothèque universitaire, puis nous l'amenons progressivement vers une architecture distribuée. Chaque jour ajoute une couche de complexité réaliste.

Mode enfant 🧸

C'est comme construire une ville. Au début, un seul immeuble fait tout (mairie, école, épicerie). Quand la ville grandit, on sépare les bâtiments, on construit des routes entre eux, on embauche du personnel spécialisé. Mais on le fait *progressivement*, pas du jour au lendemain.

Les services du projet

```
campus-library/  
├─ catalog-service      # Gestion du catalogue de livres  
├─ loan-service         # Règles d'emprunt et retours  
├─ identity-service     # Authentification, rôles (JWT)  
├─ notification-service # Emails, alertes (asynchrone)  
└─ api-gateway          # Point d'entrée unique, routage
```

Progression jour par jour

- 1 **Jour 1** — Monolithe bien structuré : couches, DTO, validation Junior
- 2 **Jour 2** — Patterns applicatifs : hexagonal, CQRS, DDD léger Intermédiaire
- 3 **Jour 3** — Systèmes distribués : réseau, CAP, idempotence, API Intermédiaire
- 4 **Jour 4** — Microservices : découpage, communication, Kafka Avancé
- 5 **Jour 5** — Résilience, observabilité, sécurité Avancé
- 6 **Jour 6** — Cloud, déploiement, gouvernance, ADR Expert

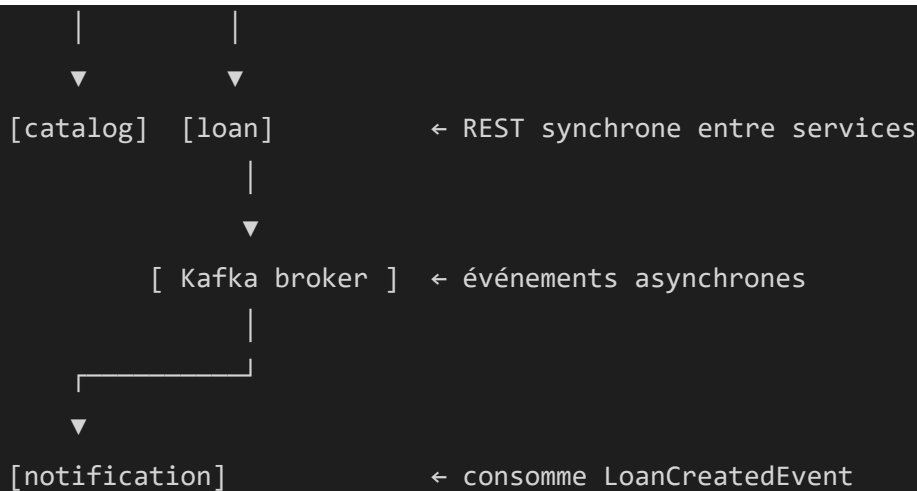


Live coding du Jour 1

Créer une première API Spring Boot propre avec couches, DTO, validation Bean et gestion d'erreurs globale (`@ControllerAdvice`). Ce sera la base du TP 1.

Schéma global de l'architecture cible

```
[ Client web / mobile ]  
  | HTTPS  
  ▼  
[ API Gateway ] ← JWT validation, rate limiting, routing
```



Architecture Decision Records (ADR)

Un **ADR** est un court document qui capture une décision architecturale importante, le contexte dans lequel elle a été prise, les alternatives considérées et les conséquences attendues. C'est de la documentation légère mais durable.

Pourquoi écrire des ADR ?

- Dans 6 mois, personne ne se souvient *pourquoi* on a choisi Kafka plutôt que RabbitMQ.
- Un nouveau développeur peut comprendre les contraintes historiques sans ré-ouvrir tous les débats.
- Forcer l'explicitation d'un choix améliore la qualité de la décision elle-même.

Exemple d'ADR — choix de PostgreSQL

```
# ADR-001 : Utilisation de PostgreSQL comme base de données principale

## Statut
Accepté — 2025-09-01

## Contexte
Le service loan-service doit gérer des transactions ACID
(un emprunt ne doit pas être créé deux fois, le stock doit
```

être décrémenté de façon atomique). La volumétrie attendue est de ~50 000 emprunts/an.

Décision

Utiliser PostgreSQL 16 via Spring Data JPA / Hibernate.

Alternatives considérées

- MongoDB : rejeté – pas de transactions multi-documents natives en v5
- MySQL 8 : acceptable, mais PostgreSQL offre de meilleures fonctionnalités JSON et de meilleurs plans de requêtes

Conséquences

- + Transactions ACID garanties
- + Écosystème Spring mature
- Schéma à migrer via Flyway à chaque évolution du modèle

✓ **Format recommandé (Michael Nygard)**

Titre · Statut · Contexte · Décision · Alternatives · Conséquences. Les ADR sont versionnés avec le code dans `docs/adr/`. Un ADR ne se modifie pas, il se *supercède* par un nouveau.

Dette technique — comprendre pour mieux décider

La **dette technique** est la somme des compromis pris volontairement (ou non) dans le code et l'architecture qui *coûteront plus cher plus tard* à rembourser qu'à éviter.

Mode enfant 🐼

C'est comme emprunter de l'argent. Parfois c'est utile (livrer vite un MVP), mais si on n'y fait pas attention, les intérêts s'accumulent et on passe tout son temps à rembourser au lieu d'avancer.

⚠ **Dette délibérée**

On choisit de ne pas faire proprement *maintenant* pour aller vite. Acceptable si on planifie le remboursement.

Exemple : copier-coller une fonction en attendant d'extraire un service commun.

✗ Dette accidentelle

On ne sait pas qu'on accumule de la dette. Souvent dûe à un manque de connaissances ou d'expérience.

Exemple : mettre toute la logique dans le contrôleur sans le savoir.

Mode pro 🎓 — Quadrant de la dette (Fowler)

	Délibéré	Accidentel
Imprudent	"On n'a pas le temps de faire propre"	"C'est quoi le pattern Repository ?"
Prudent	"On livre vite et on refactorise après"	"Maintenant on sait ce qu'on aurait dû faire"

Seule la dette **délibérée/prudente** est acceptable — et seulement si elle est documentée et remboursée.

Méthode pédagogique

Élément	Dans le cours	Pourquoi
 Mode enfant	Analogies du quotidien	Comprendre l'intuition sans jargon
 Mode pro	Enjeux d'entreprise, standards	Situer les décisions dans le réel

Élément	Dans le cours	Pourquoi
 Code commenté	Extraits Spring Boot complets	Lier théorie et pratique immédiatement
 Erreurs fréquentes	Anti-patterns nommés	Apprendre des erreurs des autres
 Best practices	Règles opérationnelles	Avoir des réflexes solides
 Mini quiz	À chaque fin de chapitre	Consolider par la réflexion active
 ADR	Décisions documentées	Comprendre le "pourquoi" des choix
 TP progressifs	TP1 → TP2 → TP3	Montée en complexité maîtrisée



Structure des TP

- **TP1** — Construire un monolithe propre : API REST, couches, validation, gestion d'erreurs.
- **TP2** — Évoluer vers le distribué : découper en 2 services, communication REST, Feign.
- **TP3** — Résilience & observabilité : circuit breaker, tracing, métriques, Kafka.

Parcours de montée en compétence

Voici comment le contenu de ce module se positionne dans le parcours global d'un développeur backend.



Junior

- Séparer contrôleur / service / repository.
- Utiliser les DTO, ne pas exposer les entités.
- Valider les entrées avec Bean Validation.
- Lire et écrire des tests unitaires.

● Intermédiaire

- Appliquer l'architecture hexagonale.
- Comprendre les compromis CAP.
- Concevoir des APIs versionnées.
- Gérer timeouts, retries, idempotence.

● Avancé

- Découper par domaine métier (DDD).
- Mettre en place CQRS / Event Sourcing.
- Circuit breaker, bulkhead, observabilité.
- Publier et consommer des événements Kafka.

● Expert

- Choisir une topologie de déploiement (cloud-native).
- Gérer la gouvernance des APIs à l'échelle.
- Écrire et défendre des ADR en comité.
- Anticiper la loi de Conway dans le découpage.

Quiz d'entrée — testez vos intuitions

Pas de panique si vous ne savez pas — ce sont précisément les questions auxquelles le module va répondre.

1. Pourquoi un monolithe n'est-il pas forcément un mauvais choix ?

► [Voir la réponse](#)

2. Quelle est la différence entre architecture et design détaillé ?

[▶ Voir la réponse](#)

3. Pourquoi la latence réseau change-t-elle la façon de coder ?

[▶ Voir la réponse](#)

4. À quoi sert un dossier d'architecture / un ADR ?

[▶ Voir la réponse](#)

5. Qu'est-ce que la Loi de Conway et pourquoi un architecte doit-il la connaître ?

[▶ Voir la réponse](#)

6. Qu'est-ce que la dette technique délibérée/prudente ?

[▶ Voir la réponse](#)[← Accueil](#)[Jour 1 →](#)